StatefulDiscovery: Evidence-Calibrated Claim Formation in Open-Ended Scientific Discovery

Jiayao Chen[1] Shi Liu[1] Linyi Yang[1,†]

[1] Southern University of Science and Technology

Open-ended scientific discovery asks agents to move beyond executing analyses for predefined questions. Across multiple rounds of exploration, a discovery agent must decide which phenomena warrant investigation while avoiding overinterpretation, where emerging claims exceed the evidential scope of the analyses supporting them. This creates an evidence-calibration problem: the exploration trajectory must be coupled with claim status so that evidence can guide both what to investigate next and what can be claimed. We introduce StatefulDiscovery, a discovery framework that externalizes investigation state and uses it to coordinate frontier selection, evidence acquisition, and claim adjudication. We evaluate StatefulDiscovery across 40 real-data discovery tasks. Compared with several baselines, StatefulDiscovery produces more claims overall judged to be both well-supported and high-value. Ablations indicate that structured hypotheses, local adjudication, and frontier control contribute to performance. Together, these results suggest that explicit discovery state can couple exploration with evidence-calibrated claim formation.

Code: `https://github.com/Nazzyyj/StatefulDiscovery-Dev`

Correspondence: `12531182@mail.sustech.edu.cn` `yangly6@sustech.edu.cn`

† Corresponding author.

1 Introduction

Autonomous research agents increasingly automate complex scientific workflows, including literature synthesis, hypothesis generation, experiment planning, and paper writing (Zhou et al., 2026, Ghareeb et al., 2026, Gottweis et al., 2026, Lu et al., 2026, Weng et al., 2025, Mitchener et al., 2025b). However, most existing systems operate within a goal-driven paradigm, anchored by predefined targets such as a specific disease mechanism or a benchmark metric. In contrast, open-ended data-driven discovery presents a more unconstrained challenge (Agarwal et al., 2025): given a dataset and broad scientific context, the agent must autonomously identify which phenomena warrant investigation. The challenge in this setting lies not merely in generating executable code for a specific analytical query, but in *evidence calibration*, determining whether emerging claims are rigorously justified by the collected evidence.

This challenge exposes a fundamental tension in evidence-based claim formation: while highly interpretive claims can offer greater scientific value, they demand correspondingly stronger evidential grounding (Figure 1). For example, a descriptive claim about slower reading in dyslexic readers is directly supported by observed metrics, while a model-fitted explanation of initial reading overhead adds interpretation but remains tied to an empirical phenomenon in the dataset (Gu et al., 2024). However, an agent risks an *interpretive leap* if it asserts that a non-native language background amplifies this difficulty without establishing comparable task constraints across groups. We use *overinterpretation* (Boutron et al.,

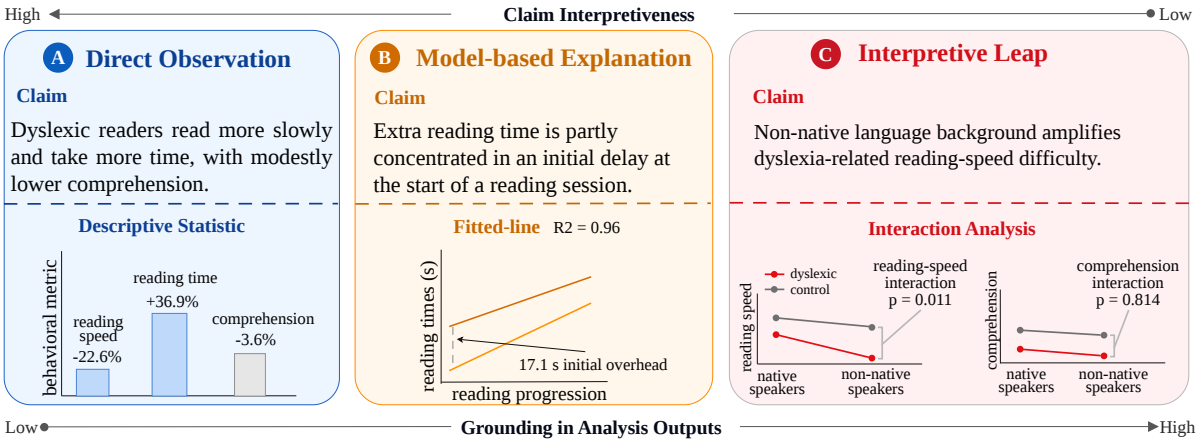


Figure 1. Support–interpretiveness trade-off in claim formation from data. (A) Descriptive claims stay close to observed metrics; (B) model-based explanations add interpretation while remaining tied to fitted evidence; (C) interpretive leaps arise when stronger claims exceed the evidential scope of the analyses.

2010, McGrath et al., 2017) to define the failure mode where the semantic strength of an agent's claim exceeds the empirical scope of its supporting analyses.

In autonomous open-ended discovery, overinterpretation is a critical vulnerability. Prior systems provide mechanisms for search, optimization, or retrospective validation (Agarwal et al., 2025, Novikov et al., 2025, Yan et al., 2026), yet they leave a crucial evidence-calibration problem structurally unresolved: **how the exploration trajectory should be coupled with the status of emerging claims?** Consequently, agents relying on implicit working memory or unstructured context windows often lose track of weak evidence, discard unresolved alternatives, and overlook evidence gaps. The necessary insight is to introduce an explicit *epistemic state*, allowing the agent to keep weakly supported claims provisional, preserve competing hypotheses, and explicitly track evidence gaps across iterations.

To this end, we present StatefulDiscovery, an autonomous discovery framework governed by an explicit epistemic state. The agent externalizes the evolving investigation state as persistent objects tracking observations, active investigations, collected evidence, and the evidential status of candidate claims. It regulates exploration through a dual-layer architecture: *global frontier control* selects the next direction to investigate, while *local hypothesis-set adjudication* compares competing hypotheses and calibrates candidate claims against the executed evidence. Together, these mechanisms systematically link frontier selection, evidence acquisition, and evidence-calibrated claim formation into a unified process.

We evaluate StatefulDiscovery on 40 real-data tasks covering biomedical, social-science, behavioral, and cross-domain datasets. To assess performance under a finite exploration budget, we introduce a novel claim-level evaluation protocol that explicitly decouples evidential support from scientific discovery value. Our empirical results demonstrate that StatefulDiscovery yields claim sets that are preferred in pairwise comparisons. Overall, it produces 23% more high-quality claims than the closest baseline, while also being preferred over that baseline in 31 of 40 pairwise comparisons. Ablation studies further confirm that structured hypothesis sets improve discovery value, while the dual-layer adjudication and control mechanisms optimize the trade-off between exploration yield and claim validity.

To our knowledge, we are the first to utilize the discovery state as a control interface for open-ended exploration. Under this framework, the status of emerging claims actively directs the agent's subsequent investigations and calibrates the confidence of its conclusions. StatefulDiscovery implements this through persistent state updates that mediate coupled decisions about frontier selection, evidence acquisition, and claim assessment across discovery rounds. We instantiate and evaluate this design in open-ended discovery tasks and introduce a claim-level evaluation protocol that separates evidential support from discovery value and compares claim formation under a finite budget.

2 Related Work

2.1 Evolutionary Research Agents

Autonomous research systems increasingly automate the scientific process (Zhou et al., 2026, Lu et al., 2026, Ghareeb et al., 2026, Gottweis et al., 2026). Frameworks such as Kosmos (Mitchener et al., 2025b), DeepScientist (Weng et al., 2025), and AutoSOTA (Li et al., 2026) further coordinate long-horizon discovery around shared contexts or benchmark targets. In parallel, evolutionary search agents iteratively propose and select candidate program variants, objectives, or computable scorers based on explicit evaluator feedback (Yan et al., 2026, Novikov et al., 2025, Du et al., 2025). However, both paradigms constrain discovery by requiring either a supplied benchmark query, a fixed target, or a scalar fitness function. To bypass these dependencies, we focus on open-ended claim formation when no specific analytical query, target claim, or optimization goal is provided in advance.

2.2 Agent Memory Management

Memory management methods help long-horizon agents store, retrieve, and organize task context through workflows, progress ledgers, hierarchical working memory, workflow traces, or learned memory policies (Wu et al., 2024, Fourney et al., 2024, Hu et al., 2024, Wang et al., 2024, Yu et al., 2026). BeliefMem (Liao et al., 2026) further argues that memory should retain multiple candidate conclusions under partial observability. StatefulDiscovery uses persistence for a more specific scientific role. The external state serves as a control interface for evidence-calibrated claim formation, not as a general memory mechanism.

2.3 Scientific Discovery Control

Existing systems guide scientific exploration in more specified settings. AutoDiscovery (Agarwal et al., 2025) prioritizes hypotheses with Bayesian surprise and tree search. PiFlow (Pu et al., 2025) and PiEvo (Pu et al., 2026) use scientific principles and uncertainty reduction. EXPERIGEN (Sengupta et al., 2026) couples hypothesis generation with experimental validation, while HypoExplore (Koo et al., 2026) maintains hypothesis memory and confidence updates for visual architecture discovery. These systems typically assume a hypothesis space, validation target, scientific principle, human-specified direction, or external feedback signal. StatefulDiscovery instead asks how accumulated evidence should guide both subsequent investigation and claim strength without a target claim or a scoring criterion in advance.

Other work develops infrastructure for autonomous science, including federated workflow agents (Kamatar et al., 2026), cooperative closed-loop discovery systems (Pauloski et al., 2025), governed drug-discovery agents (Cao et al., 2026), and auditable research frameworks (Rasheed et al., 2026). These systems can coordinate, govern, or audit discovery workflows, but they do not by themselves specify how evidence status should drive exploration and claim formation. StatefulDiscovery targets this discovery-control layer by making discovery state an interface between evidential status, frontier selection, and claim formation.

3 StatefulDiscovery: Evidence-Calibrated Claim Formation

We define StatefulDiscovery as a framework that maintains an explicit, evolving investigation state during open-ended scientific discovery. It organizes the process around bounded investigations, competing hypotheses, executable evidence, and evidence-calibrated claims. This state makes local claim formation visible to global exploration control. Figure 2 provides an overview of our agent as instantiated for data discovery.

3.1 Epistemic State and Discovery Objects

The framework represents discovery state through seven persistent discovery objects that the agent reads and updates throughout discovery. These objects correspond to the externalized-state block in Figure 2 and provide the interface between initialization, the active discovery loop, and final reporting. This design reflects a process view of scientific discovery: phenomena motivate hypotheses, hypotheses guide tests, and evidence updates what can be claimed (Whewell, 1840, Klahr and Dunbar, 1988, Langley et al., 1987, Darden, 2002).

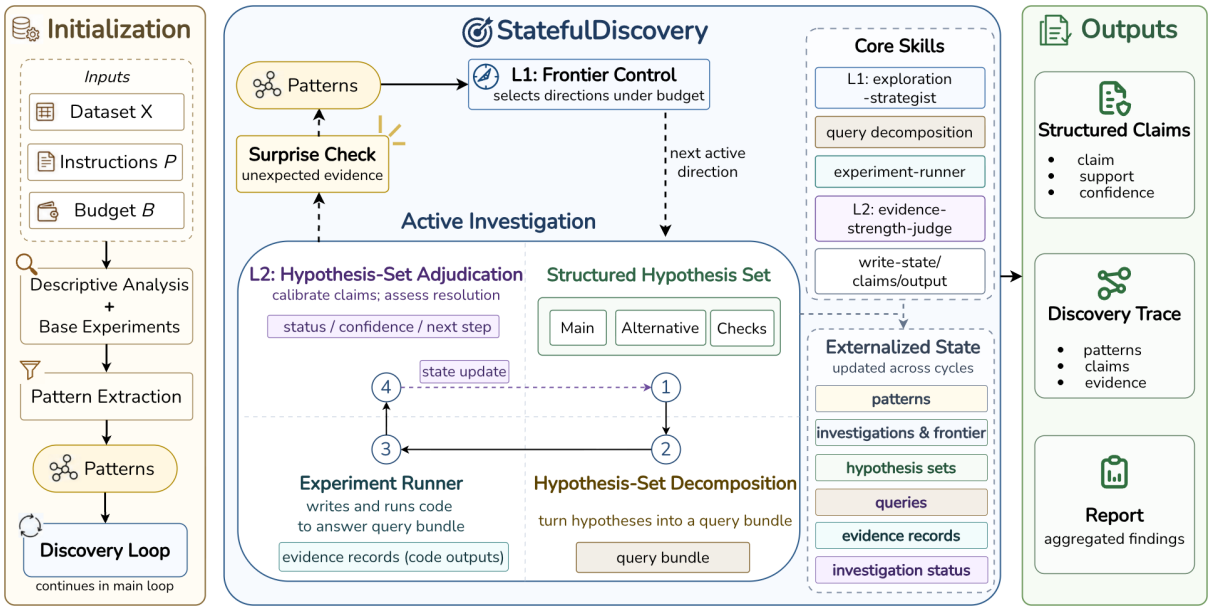


Figure 2. Overview of StatefulDiscovery. (a) Initialization uses the dataset, instructions, and budget to surface and prioritize candidate patterns. (b) The framework externalizes discovery state as persistent objects in a stateful loop: under L1 frontier control, the agent selects a direction; within an active investigation, it maintains hypotheses, runs executable analyses, and uses L2 adjudication to evaluate evidence, calibrate claim confidence, assess investigation status, and identify next steps. (c) The agent returns structured claims, a discovery trace, and an aggregated report.

A **pattern** is an observed phenomenon that may warrant further investigation. An **investigation** is a bounded line of exploration over one or more related patterns. Within each investigation, a **structured hypothesis set** includes a main hypothesis, alternatives, artifact checks, and robustness checks. A **query** is an executable analysis designed to discriminate among hypotheses, and its outputs become **evidence** records that update hypothesis and claim status. An **investigation status** summarizes what has been resolved under available evidence, while a **frontier state** records what remains worth exploring under the remaining budget.

Example discovery state.
Pattern: faster reading is associated with a larger accuracy drop in dyslexic than in controls;
Investigation: test vulnerability in dyslexic readers to speed-induced accuracy loss;
Structured hypothesis set: main speed–accuracy interaction, with page-confounding, sample-size, and speed-distribution checks;
Query: fit a dyslexia × speed model, test page interactions, bootstrap the interaction, and compare speed distributions;
Evidence: significant interaction and no page confounding, but inconclusive bootstrap robustness;
Investigation status: partially supported but robustness-limited, so the claim remains cautious;
Frontier state: carry forward a cautious claim and leave the broader reading mechanism open for later investigation.

3.2 Discovery Loop and Dual-layer Architecture

The discovery loop uses the externalized state at dual-layer architecture. As shown in Figure 2, the agent uses L1 frontier control to decide where to allocate the next unit of budget. Within the active investigation, it uses L2 local hypothesis-set adjudication to evaluate evidence, calibrate hypothesis and claim status, and update the externalized state. Before any investigation is active, initialization runs exploratory data analysis and low-cost base analyses to surface candidate patterns. The agent assigns each observed pattern an initial priority score that reflects its relative priority for further investigation. These prioritized patterns seed the initial frontier.

At the frontier level, the agent consults the externalized state to allocate effort across the broader discovery process. This L1 decision reads the existing pattern priorities together with active investigation status, frontier saturation, red flags, and remaining budget. Based on this state, the agent opens an investigation, deepens the current one, shifts to another direction, retires an unpromising investigation, or

stops. Resolved investigations release budget toward high-priority patterns, while unresolved evidence gaps keep the agent focused on the current one. When the frontier is saturated or remaining directions appear driven by artifacts, the agent may retire them or stop.

Once the agent activates an investigation through L1 frontier control, the numbered steps in Figure 2 describe the recurrent structure of the active investigation. The agent maintains a structured hypothesis set over the active direction, including a main hypothesis, alternatives, artifact checks, and robustness checks. It then translates this hypothesis set into an executable query bundle, runs the corresponding analyses, and records code outputs as evidence.

At the local level, the agent uses L2 adjudication to evaluate evidence for the active investigation. It checks whether the executed evidence answers the intended query, compares the main hypothesis against alternatives, artifact checks, and robustness checks, and assigns each hypothesis an outcome: support, weaken, refute, or inconclusive. It also records an investigation-level assessment with the best-supported explanation, remaining concerns, resolution confidence, red flags, and recommendations for next steps. This update guides whether the investigation should yield a claim, be deepened or revised, or return to frontier control.

During L2 adjudication, the agent also records a surprise signal for each executed query by comparing the query's `expected_effect`, written during hypothesis-set decomposition, with the observed evidence returned by the experiment runner. This signal is heuristic and is not a calibrated statistical test. It indicates whether the observed result contradicts or substantially departs from the expected effect. If the investigation is otherwise resolved, a surprising result can be promoted to a new pattern for the next L1 frontier decision; if the investigation remains unresolved, the signal is retained as a concern or cue in the current L2 assessment.

3.3 Skill-Based Implementation

In our implementation, skills provide the operational interface between the discovery agent and the externalized state. The discovery loop is specified by fixed skill interfaces, without a learned policy or a hand-coded scoring function. Each skill specifies which state records to read, what reasoning or analysis procedure to perform, and what structured updates to write back, following the agent-skills abstraction (Jiang et al., 2026, Shen et al., 2026).

The L1 frontier-control skill reads pattern priorities, active-investigation status, red flags, frontier saturation, and remaining budget, and returns one frontier action from a fixed set: create, attach, deepen, switch, retire, or stop. The L2 adjudication skill reads the active hypothesis set, query bundle, and executed evidence, and writes hypothesis outcomes, investigation confidence, red flags, resolution status, recommendations for next steps, surprise signals, and finalizable claims. The full skill interfaces, state schemas, and expanded read–write protocol are provided in Appendix F. Figure 16 gives the L1/L2 operational view.

4 Experiments

4.1 Experimental Setup

We evaluate StatefulDiscovery on 40 open-ended data-discovery tasks constructed from existing question-answering benchmarks. For each source task, we retain the dataset together with the minimal file and variable information needed to understand and parse the files. We remove the benchmark-specific question, target answer, and concrete analytical query. With only file paths, available variable descriptions, and a fixed budget as input, each agent is instructed to produce evidence-supported claims without a target claim to verify.

The evaluation set contains 21 tasks from BixBench[1] (Mitchener et al., 2025a), 13 from BLADE (Gu et al., 2024), and 6 from DiscoveryBench (Majumder et al., 2025), covering biomedical, social-science, behavioral, and cross-domain datasets. Unless otherwise stated, LLM calls use Qwen3.5-plus. All agents run without web search or external literature retrieval and receive a budget of 40 code executions per task.

[1] https://huggingface.co/datasets/phylobio/BixBench-Verified-50

Additional dataset details, sandbox setup, and instruction templates are provided in Appendix A.

4.2 Evaluation Metrics

Open-ended discovery requires evaluating both whether a claim is evidentially supported and whether it adds useful understanding (Whewell, 1840, Darden, 2002). We therefore decouple claim evaluation into two axes. *Evidential Support* (ES) measures whether the executed analysis output justifies a claim. *Discovery Value* (DV) measures whether the claim adds nontrivial explanatory or structural understanding beyond a uniform first-pass analysis for the same task. Both metrics are scored on a 1–5 scale using rubric-based LLM judging (Liu et al., 2023, Zheng et al., 2023, Dubois et al., 2024). The first-pass analysis is fixed per task and shared across all evaluated agents. Formally, let ρ_{ES} and ρ_{DV} denote the ES and DV judge prompts, respectively. For task i, claim c_{ij}, associated evidence e_{ij}, dataset description d_i, and first-pass analysis results f_i, we compute

$$s_{ij}^{\mathrm{ES}} = \mathrm{LLM}(\rho_{\mathrm{ES}}; d_i, c_{ij}, e_{ij}), \tag{1}$$

$$s_{ij}^{\mathrm{DV}} = \mathrm{LLM}(\rho_{\mathrm{DV}}; d_i, c_{ij}, f_i), \tag{2}$$

where $s_{ij}^{\mathrm{ES}}, s_{ij}^{\mathrm{DV}} \in \{1, 2, \ldots, 5\}$.

Before judging, outputs from all methods are normalized into a common claim–evidence format. For each claim, we extract the claim text and its supporting evidence or analysis output, excluding method-specific intermediate artifacts. To avoid favoring systems that produce more claims, ES and DV are averaged first within each task and then across tasks. Our main composite claim-quality measure is high-quality claims (HQ), defined as final claims with both ES ≥ 4 and DV ≥ 4. We also compare each method's normalized final claim set pairwise, using position-swapped judging and treating disagreements as ties. All LLM-based judges use Gemini-3.1-pro, with prompt templates provided in Appendix D.

4.3 Baselines

We compare StatefulDiscovery against four baselines that can operate under the same open-ended data-discovery protocol: a raw discovery agent, AutoDiscovery (Agarwal et al., 2025), OpenEvolve[2], and SAGA (Du et al., 2025). To reduce prompt-induced confounds, all methods receive the same dataset files, available variable descriptions, budget, and open-ended instruction to produce evidence-supported claims. Because directly comparable open-ended baselines are limited, we adapt each method to this input–output protocol while preserving its core mechanism. The baseline instructions and adaptation prompt templates are provided in Appendix C. All agents use the same Qwen3.5-plus backbone.

- **Raw agent** is a direct coding-agent baseline that uses the same backbone model and discovery environment as StatefulDiscovery, but receives only an analysis-and-report instruction. It does not use persistent discovery objects, L1 frontier control, or L2 adjudication.
- **AutoDiscovery** (Agarwal et al., 2025) is a prior open-ended discovery method based on Bayesian surprise from LLM samples and MCTS. We use the authors' open-source implementation.
- **OpenEvolve** is an open-source evolutionary coding framework based on the AlphaEvolve paradigm (Novikov et al., 2025). We adapt it to open-ended discovery by cold-starting each run with an initial discovery direction and executable analysis program. Before each code mutation, the model proposes a concrete hypothesis or analysis direction, and the mutated program is scored by an LLM judge for output validity, claim alignment, and non-triviality.
- **SAGA** (Du et al., 2025) evolves operational objectives and scorers for scientific optimization. In our setting, candidates are evidence-backed findings produced by executable analyses instead of directly mutable objects such as molecules or DNA sequences. We therefore implement candidate evolution at the analysis level: at each iteration, SAGA proposes discovery objectives, constructs or selects scorers, and scores the current finding population. High-scoring findings condition subsequent analyses, while newly generated findings are merged into the population and re-ranked for the next iteration.

[2] https://github.com/algorithmicsuperintelligence/openevolve

Table 1. Main results on the 40-task open-ended discovery evaluation set. ES/DV are task-level means with standard deviations. Claims counts final claims. HQ reports claims with both $ES \geq 4$ and $DV \geq 4$ as count/rate. Bold marks the best value in each block; ES/DV stars denote one-sided paired Wilcoxon significance against all other agents in the block, using the weakest pairwise comparison ($^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$).

Task source	Agent	ES ↑	DV ↑	Claims	HQ ↑
BixBench (Mitchener et al., 2025a)	Raw agent	4.27 ± 0.55	2.51 ± 0.59	212	29/13.7%
	AutoDiscovery	3.35 ± 1.17	2.47 ± 0.66	198	18/9.1%
	OpenEvolve	**4.74 ± 0.42****	1.31 ± 0.39	192	1/0.5%
	SAGA	3.96 ± 0.75	2.86 ± 0.95	109	**37/33.9%**
	Ours	3.75 ± 0.76	**3.14 ± 0.89**	136	33/24.3%
BLADE (Gu et al., 2024)	Raw agent	4.11 ± 0.64	2.33 ± 0.35	115	11/9.6%
	AutoDiscovery	4.42 ± 1.04	2.63 ± 0.47	130	11/8.5%
	OpenEvolve	**4.80 ± 0.34**	1.44 ± 0.41	91	3/3.3%
	SAGA	3.48 ± 1.11	2.37 ± 0.85	96	11/11.5%
	Ours	3.95 ± 0.92	**2.86 ± 0.60**	86	**22/25.6%**
DiscoveryBench (Majumder et al., 2025)	Raw agent	4.26 ± 0.31	2.61 ± 0.67	54	7/13.0%
	AutoDiscovery	3.39 ± 1.40	2.90 ± 0.29	59	7/11.9%
	OpenEvolve	**4.91 ± 0.12**	1.92 ± 0.55	68	**12**/17.6%
	SAGA	3.90 ± 1.31	2.59 ± 0.37	45	4/8.9%
	Ours	3.41 ± 1.14	**3.39 ± 0.41***	39	9/**23.1%**
Overall	Raw agent	4.22 ± 0.54	2.47 ± 0.53	381	47/12.3%
	AutoDiscovery	3.70 ± 1.24	2.59 ± 0.57	387	36/9.3%
	OpenEvolve	**4.78 ± 0.36*****	1.44 ± 0.46	351	16/4.6%
	SAGA	3.80 ± 1.04	2.66 ± 0.91	250	52/20.8%
	Ours	3.76 ± 0.87	**3.09 ± 0.76***	261	**64/24.5%**

5 Results

Table 1 summarizes the main comparison on the 40-task evaluation set, and Figure 3 shows the corresponding claim-level score distributions. StatefulDiscovery achieves the highest overall Discovery Value (3.09) and the highest observed high-quality claim count and rate (64 claims, 24.5%), while SAGA is the closest baseline on HQ (52 claims, 20.8%).

5.1 Open-ended Scientific Discovery

StatefulDiscovery also produces fewer claims than the raw agent and AutoDiscovery, suggesting that its support–value gains are not simply due to larger output volume. OpenEvolve illustrates why separating ES from DV is useful: it obtains the highest ES (4.78), but the lowest DV (1.44) and HQ rate (4.6%), with Figure 3 showing that its discovery-value scores concentrate below the high-score threshold.

A coarse manual inspection of OpenEvolve's final claims supports this interpretation. Of its 351 deduplicated claims, 74 (21.1%) are data-availability or format-check statements. Removing these statements changes the overall scores only modestly, from ES = 4.78 and DV = 1.44 to ES = 4.77 and DV = 1.61, indicating that the low DV is not mainly driven by file- or format-level claims. Among the remaining claims, 189 of 277 (68.2%) are simple pairwise associations or descriptive/statistical summaries, suggesting a broader tendency toward evidence-aligned but local statistical claims.

Table 2 provides a complementary claim-set evaluation. StatefulDiscovery is preferred over the raw agent on 25 of 40 tasks, over AutoDiscovery on 32, over OpenEvolve on all 40, and over SAGA on 31. This complements the claim-level metrics. SAGA is strong on individual claims, especially on BixBench, but pairwise evaluation favors StatefulDiscovery when judging the final claim set as a whole, including whether claims are mutually consistent and calibrated to the evidence. A GPT-5.5 second-judge check preserves the same qualitative pattern (Appendix B.2), and a length analysis indicates that the gains are not explained by longer claim-set presentations (Appendix B.3). Besides, Appendix B.1 reports efficiency and cost, Appendix H provides a qualitative case study, and figure 19 illustrates a complete trace from the a discovery task.

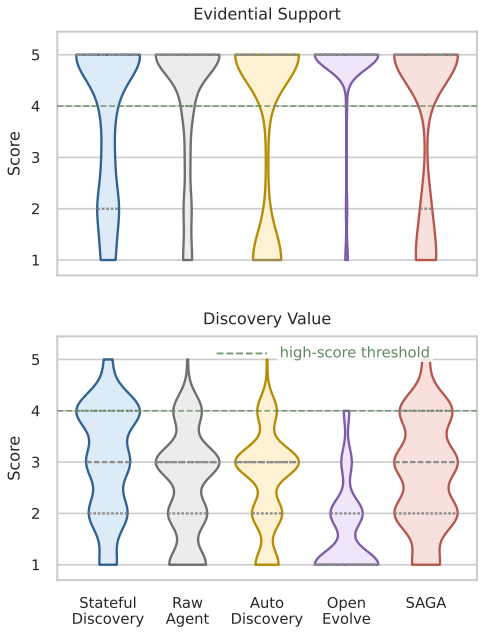


Figure 3. Claim-level score distributions across all 40 tasks. The dashed line marks the high-score threshold used in the HQ metric (ES ≥ 4 and DV ≥ 4).

Table 2. Pairwise judge preferences on the shared 40-task evaluation set.

Comparison	Win	Tie	Loss
Ours vs. Raw agent	25	6	9
Ours vs. AutoDiscovery	32	6	2
Ours vs. OpenEvolve	40	0	0
Ours vs. SAGA	31	7	2

5.2 Human Validation of Automatic Scores

We further validate the automatic claim-level scores against human annotations. For each metric, we sample 120 claims using stratified sampling over automatic judge scores and the five evaluated systems. The sample is balanced across systems and distributed across tasks to avoid concentrating the validation set on a small number of datasets. Two PhD-level annotators from different institutions, with backgrounds in statistics and biology, independently score the sampled claims using the same 1–5 rubrics. Table 3 reports within-one agreement, mean absolute error, and rank correlation for human–human agreement and for the automatic judge against the mean human score. The automatic judge shows reasonable consistency with expert annotations, supporting its use as a scalable proxy for aggregate system-level comparisons.

Table 3. Human validation of automatic claim-level scores. Agreement is reported for Discovery Value (DV) and Evidential Support (ES) using within-one agreement, mean absolute error (MAE), and Spearman rank correlation.

Metric	Comparison	Within 1	MAE	Spear. ρ
DV	Human A vs. Human B	98.3%	0.233	0.860
DV	Human mean vs. Judge	91.7%	0.617	0.782
ES	Human A vs. Human B	96.6%	0.389	0.809
ES	Human mean vs. Judge	86.4%	0.627	0.687

5.3 Ablation Study

Table 4 isolates the framework components in a cumulative ablation. Pattern recognition adds persistent pattern records, priorities, and priority-ordered follow-up experiments. The structured-hypothesis-set variant groups related patterns into bounded investigations with main, alternative, artifact, and robustness

roles, and decomposes each investigation into executable query bundles, but does not adjudicate claim status. L2 adjudication adds local evidence-strength judging and per-claim status updates for each active investigation. The full system adds L1 frontier control and an externalized state for selecting whether to create, deepen, switch, retire, or stop investigations. Because these components are sequentially dependent, each row adds one component to the previous configuration.

Table 4. Cumulative ablation results. ES and DV are task-level means with standard deviations. Claims and HQ follow the same definitions as Table 1; the Raw agent and StatefulDiscovery rows reuse the Overall rows from the main comparison.

Method	ES ↑	DV ↑	Claims	HQ ↑
Raw agent	**4.22 ± 0.54**	2.47 ± 0.53	381	47/12.3%
+ Pattern	3.52 ± 1.04	2.63 ± 0.64	283	39/13.8%
+ Hypothesis-set	3.18 ± 1.56	**3.23 ± 0.71**	277	62/22.4%
+ L2	3.63 ± 1.33	3.11 ± 1.02	142	38/**26.8%**
+ L1 (ours)	3.76 ± 0.87	3.09 ± 0.76	261	**64**/24.5%

The ablation highlights the trade-off among support, value, and yield. The raw agent obtains the highest ES, but its DV and HQ rate remain low, suggesting that evidence-aligned claims are often shallow. Pattern and hypothesis-set structure create more valuable investigation directions: the structured-hypothesis variant reaches the highest mean DV, but also lowers ES because more interpretive claims require calibration. L2 adjudication then filters unresolved or weakly supported claims, raising the HQ rate while reducing output volume. Adding L1 frontier control restores exploration yield under the same adjudication layer, maintaining similar ES and DV to the L2-only variant while producing the largest number of high-quality claims.

5.4 Backbone Model Sensitivity Analysis

We also test whether the framework depends on a specific backbone by running the same agent design with three models on six shared tasks: two from BLADE, one from DiscoveryBench, and three from BixBench. Figure 4 reports this small-scale sensitivity analysis. GPT-5.3-codex and Claude-Opus-4.6 generally improve ES, suggesting stronger evidence alignment. DV is more mixed: Claude-Opus-4.6 achieves the strongest or tied-strongest DV on four tasks, while Qwen3.5-plus remains competitive on two BixBench cases. These results suggest that the framework is not tied to a single backbone: stronger models can improve executable analysis and evidence alignment, but high-value discoveries also require effective exploration choices and hypothesis organization.

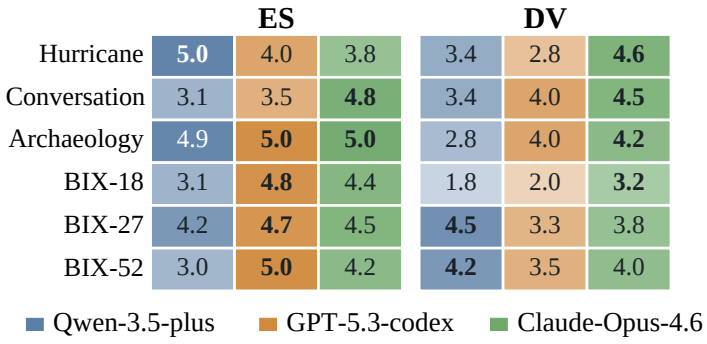


Figure 4. Backbone sensitivity on six shared tasks. Each cell reports the task-level mean ES or DV score for one backbone, using the same judges as in the main evaluation. Rows include two BLADE tasks, one DiscoveryBench task, and three BixBench tasks: BIX-18, BIX-27, and BIX-52. Darker shading indicates higher scores.

6 Conclusion

We introduced StatefulDiscovery, an autonomous discovery framework that makes discovery state an explicit control interface for open-ended scientific discovery. The agent maintains observations, hypotheses, evidence, candidate claims, and unresolved frontier directions across discovery rounds, allowing accumulated evidence to shape both what to investigate next and how strongly emerging claims are reported. In open-ended data discovery across 40 real-data tasks, StatefulDiscovery achieves the highest observed overall DV and HQ count/rate, and is preferred in pairwise final claim-set comparisons over raw coding, surprise-guided discovery, evolutionary program search, and objective-evolving optimization baselines. Ablations further show that structured hypotheses and dual-layer architecture contribute to the quality–yield trade-off. These results suggest that explicit discovery state is useful for building scientific agents that produce evidence-calibrated and scientifically valuable claims.

References

Dhruv Agarwal, Bodhisattwa Prasad Majumder, Reece Adamson, Megha Chakravorty, Satvika Reddy Gavireddy, Aditya Parashar, Harshit Surana, Bhavana Dalvi Mishra, Andrew McCallum, Ashish Sabharwal, and Peter Clark. Autodiscovery: Open-ended scientific discovery via bayesian surprise. In *Advances in Neural Information Processing Systems*, 2025.

Isabelle Boutron, Susan Dutton, Philippe Ravaud, and Douglas G. Altman. Reporting and interpretation of randomized controlled trials with statistically nonsignificant results for primary outcomes. *JAMA*, 303(20):2058–2064, 2010. doi: 10.1001/jama.2010.651.

He Cao, Siyu Liu, Fan Zhang, Zijing Liu, Hao Li, Bin Feng, Shengyuan Bai, Leqing Chen, Kai Xie, and Yu Li. Mozi: Governed autonomy for drug discovery LLM agents, 2026. URL https://arxiv.org/abs/2603.03655.

Lindley Darden. Strategies for discovering mechanisms: Schema instantiation, modular subassembly, forward/backward chaining. *Philosophy of Science*, 69(S3):S354–S365, 2002. doi: 10.1086/341858.

Yuanqi Du, Botao Yu, Tianyu Liu, Tony Shen, Junwu Chen, Jan G. Rittig, Kunyang Sun, Yikun Zhang, Aarti Krishnan, Yu Zhang, et al. Accelerating scientific discovery with autonomous goal-evolving agents, 2025. URL https://arxiv.org/abs/2512.21782.

Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. Length-controlled alpacaeval: A simple way to debias automatic evaluators. In *COLM*, 2024.

Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-One: A generalist multi-agent system for solving complex tasks, 2024. URL https://arxiv.org/abs/2411.04468.

Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J. Szostkiewicz, Daniel Vdovic, Hansol Kim, Michaela Hinks, Michael Retchin, Tyler He, Jon Laurent, Sheng Zhang, Quentin L. Gallardo, Mark W. Sellmyer, Drew M. Weissman, Andrew White, and Samuel G. Rodriques. A multi-agent system for automating scientific discovery. *Nature*, 2026. doi: 10.1038/s41586-026-10652-y. URL https://doi.org/10.1038/s41586-026-10652-y.

Juraj Gottweis et al. Accelerating scientific discovery with co-scientist. *Nature*, 2026. doi: 10.1038/s41586-026-10644-y. URL https://doi.org/10.1038/s41586-026-10644-y.

Ken Gu, Ruoxi Shang, Ruien Jiang, Keying Kuang, Richard-John Lin, Donghe Lyu, Yue Mao, Youran Pan, Teng Wu, Jiaqian Yu, Yikun Zhang, Tianmai M. Zhang, Lanyi Zhu, Mike A Merrill, Jeffrey Heer, and Tim Althoff. BLADE: Benchmarking language model agents for data-driven science. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 13936–13971, Miami, Florida, USA, November

2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.815. URL https://aclanthology.org/2024.findings-emnlp.815/.

Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. HiAgent: Hierarchical working memory management for solving long-horizon agent tasks with large language model, 2024. URL https://arxiv.org/abs/2408.09559.

Zhengyu Hu, Linxin Song, Jieyu Zhang, Zheyuan Xiao, Tianfu Wang, Zhengyu Chen, Nicholas Jing Yuan, Jianxun Lian, Kaize Ding, and Hui Xiong. Explaining length bias in LLM-based preference evaluations. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 6763–6794, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.358. URL https://aclanthology.org/2025.findings-emnlp.358/.

Kexin Huang, Serena Zhang, Hanchen Wang, Yuanhao Qu, Yingzhou Lu, Yusuf Roohani, Ryan Li, Lin Qiu, Junze Zhang, Yin Di, et al. Biomni: A general-purpose biomedical AI agent. *bioRxiv*, pages 2025–05, 2025.

Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. Sok: Agentic skills – beyond tool use in llm agents, 2026. URL https://arxiv.org/abs/2602.20867.

Alok Kamatar, J. Gregory Pauloski, Yadu Babuji, Ryan Chard, Mansi Sakarvadia, Daniel Babnigg, Kyle Chard, and Ian Foster. Empowering scientific workflows with federated agents, 2026. URL https://arxiv.org/abs/2505.05428.

David Klahr and Kevin Dunbar. Dual space search during scientific reasoning. *Cognitive Science*, 12(1): 1–48, 1988. doi: 10.1207/s15516709cog1201_1.

Jaywon Koo, Jefferson Hernandez, Ruozhen He, Hanjie Chen, Chen Wei, and Vicente Ordonez. Agentic discovery with active hypothesis exploration for visual recognition, 2026. URL https://arxiv.org/abs/2604.12999.

Pat Langley, Herbert A. Simon, Gary L. Bradshaw, and Jan M. Zytkow. *Scientific Discovery: Computational Explorations of the Creative Processes*. MIT Press, Cambridge, MA, 1987.

Yu Li, Chenyang Shao, Xinyang Liu, Ruotong Zhao, Peijie Liu, Hongyuan Su, Zhibin Chen, Qinglong Yang, Anjie Xu, Yi Fang, Qingbin Zeng, Tianxing Li, Jingbo Xu, Fengli Xu, Yong Li, and Tie-Yan Liu. Autosota: An end-to-end automated research system for state-of-the-art ai model discovery, 2026. URL https://arxiv.org/abs/2604.05550.

Junfeng Liao, Qizhou Wang, Jianing Zhu, Bo Du, Rui Yan, and Xiuying Chen. BeliefMem: Agent memory under partial observability, 2026. URL https://arxiv.org/abs/2605.05583.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522, 2023.

Chris Lu, Cong Lu, Robert Tjarko Lange, Yutaro Yamada, Shengran Hu, Jakob Foerster, David Ha, Jeff Clune, et al. Towards end-to-end automation of AI research. *Nature*, 651:914–919, 2026. doi: 10.1038/s41586-026-10265-5.

Bodhisattwa Prasad Majumder, Harshit Surana, Dhruv Agarwal, Bhavana Dalvi Mishra, Ashish Sabharwal, and Peter Clark. Discoverybench: Towards data-driven discovery with large language models, 2025. URL https://openreview.net/forum?id=vyflgpwfJW.

Trevor A. McGrath, Matthew D. F. McInnes, Nick van Es, Mariska M. G. Leeflang, Daniël A. Korevaar, and Patrick M. M. Bossuyt. Overinterpretation of research findings: Evidence of "spin" in systematic reviews of diagnostic accuracy studies. *Clinical Chemistry*, 63(8):1353–1362, 2017. doi: 10.1373/clinchem.2017.271544.

Ludovico Mitchener, Jon M Laurent, Alex Andonian, Benjamin Tenmann, Siddharth Narayanan, Geemi P Wellawatte, Andrew White, Lorenzo Sani, and Samuel G Rodriques. Bixbench: a comprehensive

benchmark for llm-based agents in computational biology, 2025a. URL https://arxiv.org/abs/2503.00096.

Ludovico Mitchener, Angela Yiu, Benjamin Chang, Mathieu Bourdenx, Tyler Nadolski, Arvis Sulovari, Eric C. Landsness, Dániel L. Barabási, Siddharth Narayanan, Nicky Evans, et al. Kosmos: An AI scientist for autonomous discovery, 2025b. URL https://arxiv.org/abs/2511.02824.

Alexander Novikov et al. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

Siba Smarak Panigrahi, Jovana Videnović, and Maria Brbic. Heurekabench: A benchmarking framework for AI co-scientist. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=Y7xCdFuFw7.

J. Gregory Pauloski, Kyle Chard, and Ian T. Foster. Agentic discovery: Closing the loop with cooperative agents, 2025. URL https://arxiv.org/abs/2510.13081.

Yingming Pu, Tao Lin, and Hongyu Chen. PiFlow: Principle-aware scientific discovery with multi-agent collaboration, 2025. URL https://arxiv.org/abs/2505.15047.

Yingming Pu, Tao Lin, and Hongyu Chen. Principle-evolvable scientific discovery via uncertainty minimization, 2026. URL https://arxiv.org/abs/2602.06448.

Razeen A. Rasheed, Somnath Banerjee, Animesh Mukherjee, and Rima Hazra. From fluent to verifiable: Claim-level auditability for deep research agents, 2026. URL https://arxiv.org/abs/2602.13855.

Jishu Sengupta, Harini SI, Somesh Singh, Yaman K. Singla, David Doermann, Rajiv Ratn Shah, Syed Mohamad Tawseeq, and Balaji Krishnamurthy. Accelerating social science research via agentic hypothesization and experimentation, 2026. URL https://arxiv.org/abs/2602.07983.

Shuaike Shen, Wenduo Cheng, Mingqian Ma, Alistair Turcan, Martin Jinye Zhang, and Jian Ma. Skillfoundry: Building self-evolving agent skill libraries from heterogeneous scientific resources, 2026. URL https://arxiv.org/abs/2604.03964.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024. URL https://arxiv.org/abs/2409.07429.

Yixuan Weng, Minjun Zhu, Qiujie Xie, Qiyao Sun, Zhen Lin, Sifan Liu, and Yue Zhang. Deepscientist: Advancing frontier-pushing scientific findings progressively. *arXiv preprint arXiv:2509.26603*, 2025.

William Whewell. *The Philosophy of the Inductive Sciences*, volume 2. John W. Parker, London, 1840.

Yiran Wu, Tianwei Yue, Shaokun Zhang, Chi Wang, and Qingyun Wu. StateFlow: Enhancing LLM task-solving through state-driven workflows, 2024. URL https://arxiv.org/abs/2403.11322.

Minghao Yan, Bo Peng, Benjamin Coleman, Ziqi Chen, Zhouhang Xie, Shuo Chen, Zhankui He, Noveen Sachdeva, Isabella Ye, Weili Wang, Chi Wang, Ed H. Chi, Fernando Pereira, Wang-Cheng Kang, Derek Zhiyuan Cheng, and Beidou Wang. Pacevolve: Enabling long-horizon progress-aware consistent evolution, 2026. URL https://arxiv.org/abs/2601.10657.

Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents, 2026. URL https://arxiv.org/abs/2601.01885.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, 2023.

Lianhao Zhou, Hongyi Ling, Cong Fu, Yepeng Huang, Michael Sun, Wendi Yu, Xiaoxuan Wang, Xiner Li, Xingyu Su, Junkai Zhang, et al. Autonomous agents for scientific discovery: Orchestrating scientists, language, code, and physics, 2026. URL https://arxiv.org/abs/2510.09901.

A Additional Experimental Details

Task construction.. We construct 40 open-ended data-discovery tasks from existing question-answering benchmarks. For each source task, we retain the dataset files and variable descriptions needed to understand and parse the data, including file paths, variable names, column descriptions when available, and data types. We remove the benchmark-specific question, target answer, and concrete analytical query. The resulting task asks an agent to explore the dataset and produce evidence-supported claims without being given a target claim to verify.

Dataset sources.. The evaluation set draws from three publicly released benchmarks. We do not construct new data or attempt to identify individuals.

- **BixBench** (Mitchener et al., 2025a) pairs biology questions with associated datasets. We use 21 datasets from BixBench-Verified-50[3], covering genomics, transcriptomics, proteomics, epigenomics, microbiome analysis, and biomedical statistics.
- **BLADE** (Gu et al., 2024) contains realistic tabular data-analysis tasks. We use 13 datasets spanning social science, education, economics, policy, sports, behavioral data, and synthetic settings.
- **DiscoveryBench** (Majumder et al., 2025) contains hypothesis-driven discovery tasks across scientific domains. We use 6 datasets covering archaeology, socioeconomic outcomes, education and development indicators, and software-engineering research.

Execution environment.. StatefulDiscovery runs through OpenCode[4] in a Docker sandbox. For each task, the sandbox mounts the dataset files and a Markdown instruction file; Appendix G gives the instruction template used for StatefulDiscovery. All methods are evaluated without web search or external literature retrieval. Agents may use the provided scientific Python environment and install analysis packages when needed. Unless otherwise stated, we use Qwen3.5-plus as the backbone model and set the budget to 40 code executions per task. All executed code, intermediate outputs, structured claims, and final reports are saved for evaluation.

Artifact licenses and terms.. We use public datasets, software, and model APIs under their stated licenses or terms. Confirmed public license metadata includes Apache-2.0 for BixBench-Verified-50 and BLADE, MIT for OpenCode, and Apache-2.0 for OpenEvolve. Other third-party artifacts are cited at their source releases or documentation. We do not redistribute third-party datasets, model weights, or baseline code beyond links and reproduction instructions.

B Additional Experimental Results

B.1 Efficiency and Cost

Table 5 reports average wall-clock time and API cost per task as practical run-time references. We ran the methods under the same network and API service conditions, so the numbers provide an implementation-level comparison.

Table 5. Average runtime and cost per task. AutoDiscovery uses Qwen3.5-plus as the backbone model. Costs are reported in USD.

Method	Cost / task	Time / task
AutoDiscovery	$0.83	105 min
OpenEvolve	$0.58	57 min
SAGA	$0.59	73 min
Raw agent	$0.22	23 min
StatefulDiscovery	$0.55	42 min

[3] https://huggingface.co/datasets/phylobio/BixBench-Verified-50
[4] https://github.com/opencode-ai/opencode

B.2 Pairwise Judge Robustness

As a robustness check, we repeat the pairwise claim-set comparison with GPT-5.5 as an independent judge. Table 6 shows the same qualitative direction: StatefulDiscovery wins more often than it loses against every baseline, although the GPT-5.5 judge is more conservative than the Gemini-3.1-pro judge.

Table 6. Pairwise claim-set robustness across two LLM judges. Win/tie/loss counts are from StatefulDiscovery's perspective over 40 tasks. Gemini-3.1-pro is the primary judge used in the main results; GPT-5.5 is used only for this robustness check.

Comparison	Gemini-3.1-pro Win/Tie/Loss	GPT-5.5 Win/Tie/Loss
Ours vs Raw agent	25/7/8	22/9/9
Ours vs AutoDiscovery	32/7/1	30/6/4
Ours vs OpenEvolve	40/0/0	37/2/1
Ours vs SAGA	31/7/2	27/6/7

B.3 Pairwise Input Length Analysis

Prior work has shown that LLM-based preference judges can exhibit verbosity or length bias, often favoring longer outputs (Zheng et al., 2023, Dubois et al., 2024, Hu et al., 2025). We therefore measure the length of the normalized claim-set inputs used in our pairwise comparisons. The measurement uses the exact formatting function used for pairwise judging: each claim set is represented by claim text plus associated evidence. We report the number of claims per task and the number of formatted characters per task. Table 8 reports paired two-sided Wilcoxon tests against StatefulDiscovery; the descriptive means in Table 7 show the direction of the differences.

Table 7. Length statistics for normalized pairwise-judge inputs over the 40 tasks. Characters are measured after the same formatting and truncation used for pairwise judging.

Agent	Claims/task	Chars/task
StatefulDiscovery	6.53 ± 2.71	7365 ± 3610
Raw agent	9.53 ± 3.07	10480 ± 3931
AutoDiscovery	9.68 ± 0.76	11488 ± 996
OpenEvolve	8.78 ± 4.26	10063 ± 6163
SAGA	6.25 ± 2.13	10756 ± 4096

Table 8. Paired two-sided Wilcoxon tests for normalized pairwise input lengths, comparing each baseline against StatefulDiscovery.

Baseline	Claims/task p	Chars/task p
Raw agent	9.06×10^{-5}	1.70×10^{-4}
AutoDiscovery	1.96×10^{-6}	1.46×10^{-7}
OpenEvolve	0.0037	0.0310
SAGA	0.624	1.08×10^{-4}

StatefulDiscovery uses fewer claims than Raw agent, AutoDiscovery, and OpenEvolve, and a comparable number of claims to SAGA. Its formatted pairwise inputs are significantly shorter than those of all baselines under paired two-sided Wilcoxon tests. This does not rule out all length effects, but it reduces the concern that the pairwise preference results are driven by longer claim-set presentations.

B.4 Additional Query-Guided Evaluation

To situate the query-free setting, we also evaluate a query-guided variant on two fixed-query benchmarks, BixBench-Verified-50 and HeurekaBench-Lite (Panigrahi et al., 2026). We compare four agents in this setting. StatefulDiscovery-QG retains the query decomposition and L2 adjudication skills, while the

Raw agent uses the same coding environment and backbone with a direct analysis-and-answer prompt. Crow (Mitchener et al., 2025a) is a ReAct-like agent that writes code and observes execution results. Biomni (Huang et al., 2025) is a biomedical agent with domain-specific tools and retrieval-augmented know-how. Table 9 shows that StatefulDiscovery-QG performs competitively on both benchmarks, especially on BixBench-Verified-50 tasks that require multi-step tool use and precise numerical answers. The result is consistent with our framing: when a target question is specified, coding agents can often execute and verify the required analysis; the open-ended challenge lies in deciding what scientifically meaningful question to investigate when no target query is given.

Table 9. Query-guided evaluation. BixBench-V50 reports exact/range/LLM-verified answer accuracy on a curated subset of BixBench. HeurekaBench reports G-Eval mean score over open-ended single-cell questions; Biomni excludes one leakage case.

Agent	V50 Acc. ↑	Heureka ↑
Crow	70.0%	3.32
Biomni	60.0%	**3.57**
Raw agent	76.0%	3.36
StatefulDiscovery-QG	**80.0%**	3.41

C Baseline Implementation Prompts

Curly-braced fields denote task-specific inputs supplied through the task packet. The same task-specific fields are provided to all coding-agent baselines.

C.1 Raw Agent Instruction

The raw agent uses the same dataset files, variable information, coding environment, and code-execution budget as StatefulDiscovery, but receives a direct analysis-and-report instruction without persistent discovery objects, L1 frontier control, or L2 adjudication. The prompt asks the agent to explore the dataset, write executable analyses, test hypotheses with statistical evidence, and produce evidence-linked final claims.

This baseline is therefore not an intentionally under-specified prompt baseline: it is explicitly instructed to search for non-obvious findings and support them with executable statistical evidence. The difference is that it must manage exploration, evidence status, and claim formation implicitly in the model context rather than through the externalized state used by StatefulDiscovery.

C.2 OpenEvolve Adaptation Prompts

OpenEvolve is adapted as an evolutionary program-search baseline. We retain its population-based mutation and scalar selection structure, but condition program mutations on explicit discovery directions so that candidate programs remain aligned with open-ended data discovery. Figure 6 shows cleaned prompt snippets for the direction proposal, direction-conditioned mutation, and scalar fitness judging stages.

C.3 SAGA Adaptation Prompt

We keep SAGA's planner, scorer-construction, and analyzer prompt structure unchanged, and instantiate its candidate-generation interface for open-ended dataset discovery. Each run provides SAGA with the dataset, exploration budget, current objectives, and high-level goal shown in Figure 7.

Scientific Data Analysis Agent. You are a scientific data analysis agent running in an isolated container. Your goal is to explore a dataset and discover interesting, non-obvious findings.

Task inputs. The task packet provides:

- `{dataset_file_paths}`: paths to dataset files
- `{variable information}`: column descriptions, variable names, data types, and available dataset context
- `{budget.max_experiments}`: maximum number of code executions

Task. Read the task packet to understand the dataset, then explore the data and discover findings.

Guidelines.

- Write and run Python code to analyze the data.
- Save each analysis script and its results.
- Look for patterns, anomalies, relationships, and unexpected structure.
- Test hypotheses with statistical evidence.
- The task packet specifies `{budget.max_experiments}`.
- You may stop earlier if you judge that the dataset has been sufficiently explored.
- You must stop when the code-execution budget is exhausted.

Output requirements. Write two files before finishing:

1. `claims.json`: a JSON list of findings, where each claim includes a claim id, description, evidence, experiment references, and confidence score.
2. `report.md`: a summary report of the analysis and key findings.

Do not only print results to the console. Final claims must be written to the required output files.

Figure 5. Raw agent instruction template.

1. Direction proposal.
You are preparing or evolving a discovery program for a data-driven scientific discovery task. Choose one concrete, executable direction for the next experiment.
Inputs. `{dataset_file_paths}`: paths to dataset files; `{variable information}`: column descriptions, variable names, data types, and available dataset context; `{budget.max_experiments}`: maximum number of code executions; program scaffold or current program; optional metrics, improvement areas, and evolution history.
Return strict JSON. `analysis_type`, `target_files`, `hypothesis`, and `variables`.
Rules. Use only files and variables in the task context; respect file formats; avoid identifier, row-order, sample-number, or coordinate effects unless essential.

2. Direction-conditioned program generation / mutation.
You are improving a discovery program, not writing a general-purpose solver. Modify the program so it executes the selected discovery direction.
Inputs. `{dataset_file_paths}`: paths to dataset files; `{variable information}`: column descriptions, variable names, data types, and available dataset context; `{budget.max_experiments}`: maximum number of code executions; selected direction: `{selected_direction}`; program scaffold or current program; optional metrics and evolution history.
Task. Preserve task input loading, the required discovery output schema, and traceability between computed results and emitted claims. Update the direction, analysis, and claim-building logic when applicable.
Rules. Use exact data paths; choose parsing from variable information and filename extensions; prefer installed scientific Python libraries; reuse helpers; do not rewrite unrelated scaffolding.
Return. SEARCH/REPLACE diffs only.

3. Scalar fitness judging.
You are a careful judge for data-driven scientific discovery. Evaluate each candidate claim on two axes.
Inputs. `{dataset_file_paths}`: paths to dataset files; `{variable information}`: column descriptions, variable names, data types, and available dataset context; `{budget.max_experiments}`: maximum number of code executions; baseline initial analysis: `{baseline_initial_output}`; candidate program output: `{candidate_output}`; candidate claims: `{candidate_claims}`.
Axes.

- **alignment**: whether the claim is supported by the candidate program output
- **non-triviality**: whether the claim goes beyond the baseline initial analysis

Scoring rules. Score alignment low if the claim overstates, contradicts, or is not supported by the output. Score non-triviality low for descriptive statistics, simple correlations, obvious EDA findings, or findings already revealed by the baseline. Prefer non-obvious relationships, interactions, subgroup effects, or structure.
Return JSON only. One record per claim with `claim_id`, `alignment_bucket`, `non_triviality_bucket`, and short `reasoning`.

Figure 6. OpenEvolve adaptation prompt snippets.

High-level discovery goal. Explore the provided dataset and produce novel, evidence-backed scientific findings under the given exploration and experiment budget.

Task inputs. `{dataset_file_paths}`: paths to dataset files; `{variable information}`: column descriptions, variable names, data types, and available dataset context; `{budget.max_experiments}`: maximum number of code executions.

Analysis-generation inputs. High-level discovery goal; current SAGA objectives; score-selected parent findings from the archive; task inputs; analysis identifier.

Task. Return a JSON object containing a brief rationale and one standalone Python script for an executable dataset analysis.

Script requirements. Read only from the provided dataset directory, run in the local Python environment, and write a JSON list of findings to the specified output path. Each finding must contain a claim, evidence, data references, and limitations. For statistical findings, computed statistics such as effect sizes, test statistics, p-values, sample sizes, confidence intervals, and methods should appear in the claim or evidence.

Restrictions. Do not use web search, external knowledge bases, external datasets, package installation, or writes to the dataset directory. Temporary or derived files may only be written to the analysis workspace or temporary directories.

Candidate schema.

```
{
  "claim": string,
  "evidence": string or object,
  "execution_output": object,
  "data_references": {"files": list, "columns": list},
  "limitations": string
}
```

Claims are required to be supported by the computed evidence.

Figure 7. SAGA adaptation prompt template.

D Evaluation Prompt Templates

We lightly adapt the judge templates used in implementation to match the terminology of this paper. The scoring rubrics are unchanged in substance. The pairwise comparison uses the same position-swapped protocol described in the main text.

System. You are an expert scientific evaluator. Your task is to assess whether the experimental evidence supports a given claim. Be rigorous: check specific numbers, statistical tests, and reasoning logic.

User inputs.

- **Dataset:** `{dataset_description}`
- **Claim:** `{claim_text}`
- **Experimental evidence:** `{code_output}`

Question. Does the experimental evidence support this claim? Judge the logical soundness of the inference from evidence to conclusion.

Rubric.

- **5:** Evidence strongly supports the claim. The inference from data to conclusion is logically sound and statistically backed.
- **4:** Core conclusion is well-supported. Minor gaps do not undermine the main finding.
- **3:** Evidence supports the general direction, but some inferences are only partially justified by the data.
- **2:** Evidence is weak or the claim significantly overstates what the data shows.
- **1:** The claim contradicts the evidence, or there is no evidence provided.

Return only JSON: `evidential_support` as an integer from 1 to 5 and `evidential_support_rationale` in 2–3 sentences.

Figure 8. Evidential-support judging prompt.

System. You are an expert scientific evaluator. Your task is to assess the Discovery Value of the insight established by a candidate scientific claim relative to a provided first-pass analysis. Assume the claim is factually correct. Do not judge whether the claim is well-supported; evidential validity is evaluated separately.

User inputs.

- **Dataset:** `{dataset_description}`
- **First-pass analysis results:** `{baseline_results}`
- **Claim to evaluate:** `{claim_text}`

Question. Evaluate the Discovery Value of the claim relative to the first-pass results. Consider novelty relative to the first-pass analysis, explanatory depth, the new understanding established by the claim, and the scope of the insight.

Rubric.

- **5:** Very high value. The claim establishes a new organizing interpretation, structural explanation, or data-generating insight that changes how the dataset or phenomenon should be understood overall.
- **4:** High value. The claim establishes an important but more local explanatory insight, such as a confounder, boundary condition, heterogeneity pattern, or organizing relationship within part of the dataset or phenomenon.
- **3:** Moderate value. The claim adds a meaningful refinement beyond first-pass analysis, but mainly concerns a local association, subgroup difference, single analytical comparison, or incremental interpretation.
- **2:** Limited value. The claim is mostly a direct derivative, simple refinement, or obvious implication of the first-pass analysis.
- **1:** Minimal value. The claim is directly readable from the first-pass analysis, restates it, or adds no meaningful new interpretation.

Scoring rule. Score the insight conveyed by the claim, not the complexity of the method used to obtain it. Do not reward broad or ambitious wording unless the claim actually provides broad explanatory or structural insight beyond the first-pass analysis. Treat evidential support, correctness, and reproducibility as separate from this score.

Return only JSON: `discovery_value` as an integer from 1 to 5 and `discovery_value_rationale` in 2–3 sentences.

Figure 9. Discovery-value judging prompt.

System. You are a scientist who has commissioned two independent teams to analyze the same dataset. You are reading their final claim sets and deciding which team produced more valuable scientific insights.

User inputs.

- **Dataset:** `{dataset_description}`
- **Team A claim set:** `{system_a_claims}`
- **Team B claim set:** `{system_b_claims}`

Question. As a domain expert, which team's claim set would you find more valuable?

Consider.

- Does the claim set tell you something you would not have known from basic summary statistics?
- Are the conclusions supported by rigorous evidence, or are they speculative?
- Does the analysis reveal underlying mechanisms, confounders, or non-obvious relationships?
- Are there overreaching or contradictory claims?

Do **not** favor a claim set simply because it has more findings.

Return only JSON: `winner` as `A`, `B`, or `tie`, and `rationale` in 2–3 sentences.

Figure 10. Pairwise claim-set comparison prompt.

E Human Annotation Instructions

Annotators were non-author research-group colleagues, with annotation covered by existing research/service responsibilities and compensation arrangements.

E.1 Discovery Value

Human annotators used the same Discovery Value (DV) rubric as the automatic judge, rendered in a human-facing annotation form. Figure 11 reports the instructions given to annotators.

Task. Evaluate the Discovery Value (DV) of a candidate scientific claim relative to a provided first-pass analysis. Assume the claim is factually correct. Do not judge whether the claim is well-supported, statistically reliable, or reproducible; evidential validity is evaluated separately.

Annotation inputs.

- Dataset description.
- First-pass analysis results.
- Claim to evaluate.

Question. How much new scientific or analytical understanding does the claim establish beyond the first-pass analysis? Consider novelty relative to the first-pass analysis, explanatory depth, the new understanding established by the claim, and the scope of the insight.

Scoring rule. Score the insight conveyed by the claim, not the complexity of the method used to obtain it. Do not reward broad or ambitious wording unless the claim actually provides broad explanatory or structural insight beyond the first-pass analysis. Treat evidential support, correctness, and reproducibility as separate from this score.

Rubric.

- **5: Very high value.** The claim establishes a new organizing interpretation, structural explanation, or data-generating insight that changes how the dataset or phenomenon should be understood overall.
- **4: High value.** The claim establishes an important but more local explanatory insight, such as a confounder, boundary condition, heterogeneity pattern, or organizing relationship within part of the dataset or phenomenon.
- **3: Moderate value.** The claim adds a meaningful refinement beyond first-pass analysis, but mainly concerns a local association, subgroup difference, single analytical comparison, or incremental interpretation.
- **2: Limited value.** The claim is mostly a direct derivative, simple refinement, or obvious implication of the first-pass analysis.
- **1: Minimal value.** The claim is directly readable from the first-pass analysis, restates it, or adds no meaningful new interpretation.

Annotation output. Annotators provided an integer `dv_score` from 1 to 5. They could mark `uncertain` when the claim could not be confidently scored, and could provide an optional one- to two-sentence rationale.

Figure 11. Discovery Value human annotation instructions.

E.2 Evidential Support

Human annotators used the same Evidential Support (ES) rubric as the automatic judge, rendered in a human-facing annotation form. Figure 12 reports the instructions given to annotators.

Task. Evaluate whether the provided experimental evidence supports a given claim. Judge only evidence-to-claim alignment. Do not evaluate whether the claim is novel, important, or high in Discovery Value.

Annotation inputs.

- Dataset description.
- Claim to evaluate.
- Experimental evidence, such as code output, statistical results, tables, or summaries.

Question. Is the given experimental evidence sufficient to support the claim? Check whether the claim's main direction matches the evidence, whether numbers, variables, groups, and statistical tests are grounded in the evidence, and whether the evidence covers the claim's stated scope.

Scoring rule. Do not penalize a claim for low discovery value or limited scientific importance if it is well-supported by the evidence. Penalize claims that overstate the evidence, turn association into causation, generalize beyond the tested scope, ignore important negative or nonsignificant results, or report numbers, variables, or directions that conflict with the evidence.

Rubric.

- **5: Strong support.** The experimental evidence directly and sufficiently supports the claim. Key numbers, statistical tests, group comparisons, and the reasoning chain match the claim, and the claim does not noticeably exceed the evidence.
- **4: Core conclusion supported.** The claim's main conclusion is supported, but there are minor gaps or slightly strong wording that do not undermine the main finding.
- **3: Partial support.** The evidence supports the general direction, but the claim includes some extension, broader scope, or added interpretation that is only partially justified.
- **2: Weak support or clear overinterpretation.** The evidence contains some related signal, but it is insufficient for the main strength, scope, or causal/mechanistic wording of the claim.
- **1: Unsupported or contradicted.** The claim's core content is not tested, lacks direct evidence, reports inconsistent numbers or variables, or contradicts the provided evidence.

Annotation output. Annotators provided an integer `es_score` from 1 to 5. They could mark `uncertain` when the evidence was difficult to interpret, and could provide an optional one- to two-sentence rationale.

Figure 12. Evidential Support human annotation instructions.

F Control Skill Interfaces and Schemas

F.1 Skill Interfaces

Table 10 maps loop locations in Figure 2 to the concrete skills the agent can invoke. Artifact-writing skills maintain persistent objects; frontier-control and adjudication skills support L1 and L2; and decomposition and experiment-running skills support Steps 1–3. Beyond framework-level skills, the experiment runner can invoke specialized domain recipes, such as biomedical or phylogenetic analyses, when a query requires professional methods.

Table 10. Skills may be invoked at each loop location.

Loop location	Concrete skills
Initialization	**`write-patterns`**: record candidate patterns and priorities. **`write-state`**: seed the initial frontier.
L1 frontier control	**`exploration-strategist`**: choose create, attach, deepen, switch, retire, or stop. **`write-investigation`**, **`write-state`**: apply the selected action.
Steps 1–2	**`investigation-decomposition`**: maintain the hypothesis set and query bundle. **`write-investigation`**: record active requirements.
Step 3	**`experiment-runner`**: execute analyses. **`write-experiment`**: record outputs as evidence.
Step 4 / L2 adjudication	**`evidence-strength-judge`**: assess support, confidence, red flags, and next-step signals. **`write-investigation`**, **`write-claims`**, **`repro-gate-check`**, **`write-state`**: update hypothesis status, finalize claims, check reproducibility, and update state.

F.2 Write Interfaces and State Fields

Figure 16 expands the operational view in the L1 (**`exploration-strategist`**) and L2 (**`evidence-strength-judge`**) skills by showing the state artifacts read and written by each level. We then

name: `write-patterns`
description: Defines `patterns.json` for tracking unexplained data patterns.

Format.
Write to `patterns.json`; updated throughout exploration.

Schema.

```
{
  "patterns": [
    {
      "id": "P1",
      "observation": "specific pattern with data values or
     ↪  statistics",
      "source": "eda | base002 | exp003",
      "status": "unexplained | partially_explained | explained",
      "cluster_id": "C1 or null",
      "related_variables": ["column_name"],
      "priority": 0.0
    }
  ]
}
```

name: `write-investigation`
description: Defines investigation records, hypothesis sets, and query bundles.

Format.
Write to `investigations.json` and `current_investigation.json`; stores all investigations and the active one.

Schema.

```
{
  "active_investigation_id": "I1",
  "investigations": [
    {
      "id": "I1",
      "phenomenon": "observed phenomenon",
      "pattern_ids": ["P1"],
      "status": "active | paused | resolved | retired | saturated",
      "hypotheses": [
        {
          "id": "I1-h1",
          "role": "main | alternative_explanation | artifact_check |
         ↪  robustness",
          "status": "pending | supported | weakened | refuted |
         ↪  inconclusive",
          "confidence": null,
          "evidence_refs": ["exp003"],
          "remaining_concerns": []
        }
      ],
      "query_bundle": [
        {
          "id": "I1-q1",
          "hypothesis_id": "I1-h1",
          "result_ref": null,
          "status": "pending | done | failed"
        }
      ]
    }
  ]
}
```

name: `write-experiment`
description: Defines executable experiment records in `experiments.jsonl`.

Format.
Append one JSON object per query; each record is one budget unit written by the forked **`experiment-runner`**.

Schema.

```
{
  "experiment_id": "base001 | exp001",
  "phase": "base | exploration",
  "investigation_id": "I1 or null",
  "query_id": "I1-q1 or null",
  "question": "research question tested",
  "status": "success | failed | inconclusive",
  "summary": "result with key numbers",
  "code_path": "code/exp001.py",
  "artifacts": ["code/exp001_results.json"],
  "hypothesis_refs": ["I1-h1"],
  "limitations": []
}
```

name: `write-state`
description: Defines `epistemic_state.json` with L1/L2 assessment.

Format.
Write to `epistemic_state.json`; records frontier and resolution state.

Schema.

```
{
  "exploration_progress": {
    "cycle_id": 3,
    "budget_remaining": 22
  },
  "level1": {
    "frontier_status": "open | narrowing | exhausted",
    "saturation_assessment": "low | medium | high",
    "recommended_mode": "branch | deepen | switch | stop"
  },
  "level2": {
    "active_investigation_id": "I1",
    "resolution_status": "open | partially_resolved | resolved |
   ↪  inconclusive",
    "resolution_confidence": 0.0,
    "evidence_strength": "weak | moderate | strong",
    "red_flag_pressure": "low | medium | high",
    "next_best_resolution_step": "single uncertainty-reducing action"
  },
  "decision_log": [{"cycle_id": 3, "action": "deepen"}]
}
```

name: `write-claims`
description: Defines `claims.json` for aggregated final claims.

Format.
Write to `claims.json`; updated after L2 adjudication and finalization.

Schema.

```
{
  "claims": [
    {
      "claim_id": "I1-h1",
      "investigation_id": "I1",
      "role": "main | alternative_explanation | artifact_check |
     ↪  robustness",
      "text": "claim statement with key numbers",
      "status": "supported | weakened | refuted | inconclusive",
      "confidence": 0.0,
      "evidence_refs": ["exp003"],
      "supporting_stats": {}
    }
  ]
}
```

Control records. `local_adjudication` records hypothesis evaluations, confidence, red flags, and next step. `strategy_recommendation` records selected action, target, rationale, status updates, and budget assessment.

Figure 13. Write interfaces and state-field schema used by the framework skills.

name: `evidence-strength-judge`
description: Invoke after **`experiment-runner`** completes an investigation query bundle. Evaluates hypotheses, produces a local adjudication, and writes L2 state updates.

Epistemic Evidence Assessor.
Inputs. Read the active investigation, its hypothesis set and query bundle, the referenced experiment records, and the executed code/results needed to verify the evidence.

Checks.
1. **Evidence sanity:** verify sample sizes, filters, key statistics, and reported numbers; flag mismatches between code outputs and evidence records.
2. **Method fit:** check that code implements the query requirements; flag unauthorized filtering, transformations, or inconsistent preparation.
3. **Cross-hypothesis adjudication:** compare the main hypothesis against alternatives; use artifact checks and robustness checks to weaken unsupported explanations.
4. **Confidence calibration:** assign status and confidence for each hypothesis; cap confidence when red flags are present.

Decision rules. Mark a hypothesis as supported when evidence matches the hypothesis, sanity and method checks pass, and alternatives or artifact explanations are weaker. Mark it as weakened when evidence is partial, sensitive, or better explained by an alternative or artifact. Mark it as refuted when evidence contradicts the hypothesis or a competing explanation is clearly better supported. Mark it as inconclusive when evidence is insufficient, flawed, or ambiguous. Any red flag caps confidence at 0.6; unresolved useful gaps trigger deepening; methodological flaws trigger self-correction.

Confidence rubric. 0.9+ means strong evidence with no red flags and robust checks; 0.7–0.9 means reasonable evidence with minor concerns; 0.5–0.7 means uncertain evidence, red flags, or sensitivity to choices; below 0.5 means weak evidence.

Writes. Write a local adjudication control record, hypothesis status and concerns via **`write-investigation`**, the L2 update to `epistemic_state` via **`write-state`**, and finalizable supported claims via **`write-claims`**.

Figure 14. L2 evidence-adjudication skill prompt.

name: `exploration-strategist`
description: Invoke after L2 adjudication or initialization. Recommends the next frontier action by combining pattern priority, investigation status, L2 adjudication signals, and remaining budget.

Exploration Strategist.
Inputs. Read task budget, L1/L2 fields in `epistemic_state`, pattern priorities and statuses, investigation statuses, and the latest local adjudication.

Decision policy.
1. Treat L2 as the freshest signal for the active investigation: resolved high-confidence investigations can be left behind, unresolved useful gaps should be deepened, and high red-flag pressure triggers self-correction, deepening, or retirement.
2. Assess the frontier: high-priority unexplained patterns create or attach investigations, related patterns are attached to existing investigations, and saturated or low-value residue triggers switching, retirement, or stopping.
3. Apply budget awareness: low remaining budget favors finishing active investigations, while exhausted budget requires stopping.

Action matrix. For an open frontier and resolved investigation, create a new investigation or attach a pattern. For an open frontier and unresolved investigation, deepen the investigation or create a new one when priorities and budget favor it. For a narrowing frontier, create a new investigation only if high-priority patterns remain; otherwise deepen unresolved work or stop. For an exhausted frontier, stop when resolved, or deepen only if unresolved work and budget remain.

Actions. The strategy may select `create_investigation`, `attach_pattern`, `deepen_investigation`, `switch_investigation`, `retire_investigation`, or `stop`.

Writes. Write a `strategy_recommendation` control record, pattern status and priority updates via **`write-patterns`**, investigation action/status updates via **`write-investigation`**, and the L1 update to `epistemic_state` via **`write-state`**.

Figure 15. L1 frontier-control skill prompt.

record the implementation-facing fields used by the corresponding write and decision skills. Figures 14 and 15 provide the corresponding skill prompts.

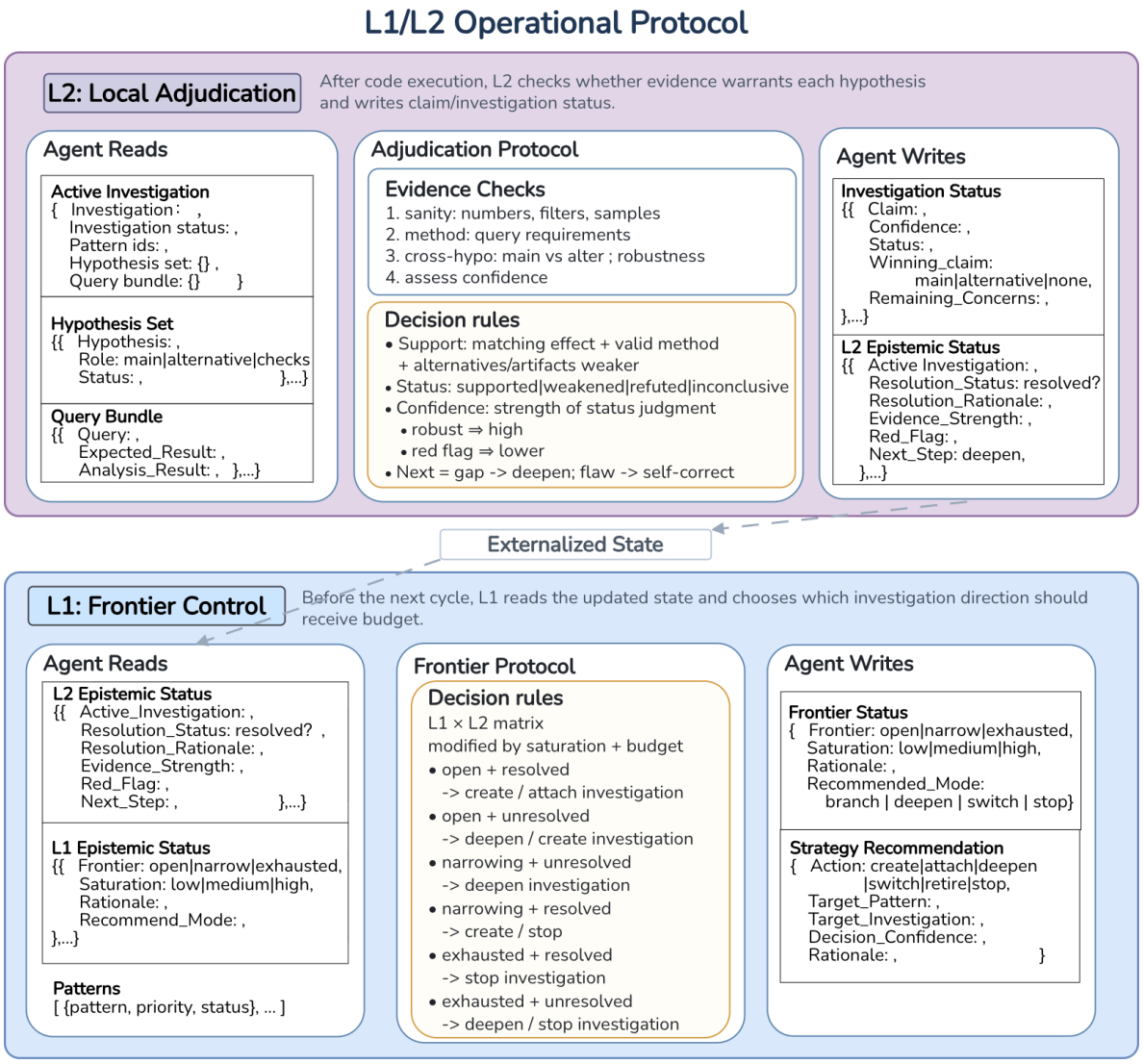


Figure 16. Expanded view of the L1/L2 operational protocol. The upper panel details how L2 reads the active investigation, checks executable evidence, and writes investigation status and L2 epistemic status. The lower panel details how L1 reads the updated state, combines resolution and frontier signals, and writes the next frontier recommendation.

G Agent Instruction Example

The task instructions are implemented as a Markdown agent-instruction file mounted in the sandbox. The released repository contains the exact file used in our experiments. Figures 17–18 provide a cleaned example in the same prompt-template style, preserving the execution flow while omitting path-level and provider-specific details.

StatefulDiscovery agent instructions: role, inputs, and initialization.

You are an autonomous scientific discovery agent running in an isolated sandbox. Given task instructions, dataset files, and a finite experiment budget, explore the dataset and produce executable-evidence-supported scientific findings.

Task inputs. The task packet provides:

- `{dataset_file_paths}`: paths to dataset files
- `{variable information}`: column descriptions, variable names, data types, and available dataset context
- `{budget.max_experiments}`: maximum number of code executions

Persistent artifacts. Maintain persistent artifacts for candidate patterns and priorities, investigations and the active investigation, query requirements and evidence records, L2 adjudication records, the externalized epistemic state, L1 strategy recommendations, final claims, and the final report.

Phase 0: Initialization.

1. Read the task instructions and dataset files.
2. Run exploratory analyses and cheap base experiments.
3. Extract data-anchored patterns; each pattern must reference evidence from base analyses and receive an agent-assigned relative priority score in $[0, 1]$ among the current patterns.
4. Record patterns with **`write-patterns`**.
5. Initialize the externalized state with **`write-state`**: frontier status open, saturation low, and no active investigation.
6. Invoke **`exploration-strategist`** to select the first investigation direction.

Figure 17. StatefulDiscovery agent instruction template: role, inputs, and initialization.

StatefulDiscovery agent instructions: exploration loop.

Each cycle follows the same rhythm.

1. Read the current strategy recommendation. Apply any pattern or investigation-status updates, then execute one frontier action: create investigation, attach pattern to an existing investigation, deepen investigation, switch investigation, retire investigation, or stop.

2. Maintain the active investigation. Invoke **`investigation-decomposition`** to maintain the structured hypothesis set, main hypothesis, alternative hypotheses, artifact checks, robustness checks, and executable query bundle. Persist the active investigation and query bundle with **`write-investigation`**.

3. Execute analyses. Invoke the forked **`experiment-runner`** to execute the query bundle. Record code outputs and evidence records with **`write-experiment`**.

4. Apply L2 local adjudication. Invoke **`evidence-strength-judge`** to evaluate the active investigation. For each hypothesis, assign one status: supported, weakened, refuted, or inconclusive. Update the active investigation with hypothesis status, confidence scores, evidence links and counter-evidence, red flags, and remaining concerns. Update the externalized state with resolution status, resolution confidence, evidence strength, red-flag pressure, and the next best resolution step. If sanity or method checks fail, revise the active investigation before advancing the frontier. Merge finalizable supported claims with **`write-claims`**.

5. Perform surprise check. Compare each query's expected result with its observed result. If the investigation is resolved and the result is surprising, add a new pattern with **`write-patterns`** for future frontier control. If the investigation remains unresolved and the result is surprising, strengthen the case for self-correction or further testing.

6. Apply L1 frontier control. Update pattern statuses with **`write-patterns`**, then invoke **`exploration-strategist`**. Choose the next action using the L2 resolution signal, pattern priorities, frontier saturation, red-flag pressure, and remaining budget. Apply the selected frontier action with **`write-investigation`** when it creates, attaches, deepens, switches, or retires an investigation. Update the externalized state with frontier status, saturation, recommended mode, budget progress, and the decision log. Repeat until the strategy recommendation is stop.

Finalization. When the strategy recommendation is stop, aggregate supported claims with **`write-claims`**, write the final report organized by investigation, and apply **`repro-gate-check`** to ensure claims link to executable evidence.

Hard constraints.

- Do not use external search or external data.
- Respect `{budget.max_experiments}`; one executed query counts as one experiment.
- Every final claim must link to executable evidence.

Figure 18. StatefulDiscovery agent instruction template: the exploration loop.

H Case Study: Surprise and Explicit State

To understand the difference behind the aggregated results, we examine the hurricane dataset in BLADE, where the relationship between hurricane-name femininity and deaths can be explained in several ways. Table 11 compares AutoDiscovery and StatefulDiscovery on this task.

Table 11. Case study on the hurricane dataset. For AutoDiscovery, numbers report the means of sampled agreement distributions before and after evidence, together with the resulting surprise/reward S; "down" indicates that evidence lowers agreement with the original hypothesis.

Aspect	AutoDiscovery	StatefulDiscovery
Representative outputs	1. U-shaped relation between name femininity and property damage (0.73 → 0.26; down; $S = 1.14$).	1. The gender–death pattern is driven by random clustering of extreme events with female names, not a causal mechanism.
	2. Low pressure and high femininity jointly increase deaths (0.27 → 0.58; supported; $S = 0.50$).	2. The apparent pattern is a time-period artifact: the 1960s used only female names and included several catastrophic storms.
	3. Quadratic relation between name femininity and deaths, with extreme names deadlier (0.73 → 0.42; down; $S = 0.50$).	3. Binary name gender reflects the historical naming convention; the femininity score is largely redundant with gender.
	4. Minimum pressure better predicts property damage than wind speed, with category moderation (0.73 → 0.42; down; $S = 0.50$).	4. Category–death associations are confounded by wind speed and inflated by a few high-death outliers.
Organization	Separate hypothesis nodes are scored by surprise; a sharp agreement shift can rank a rejected local hypothesis highly.	The apparent association is decomposed into linked investigations: gender–death causality vs. outlier/time-period alternatives, masfem naming-convention redundancy, and category/wind/outlier confounding.

The two agents discover different kinds of findings. AutoDiscovery ranks hypotheses by surprise, and in most cases the sampled agreement with the original hypothesis decreases after evidence is shown. StatefulDiscovery instead keeps the name–death association as a connected set of investigations, linking gender effects, naming conventions, storm severity, and outlier concentration. This pattern reflects how AutoDiscovery's surprise reward can be high when evidence substantially changes the agreement distribution for a hypothesis, even if the update mainly weakens that hypothesis. Such negative evidence is useful, but it does not by itself determine how related hypotheses should be retained, weakened, or connected. In this sense, Bayesian surprise in AutoDiscovery measures a model-belief update, not necessarily a scientific surprise that changes how the phenomenon is understood.

I Discovery Trace Example

Figure 19 illustrates one complete trace from the BIX-52 task. The example shows how the initial pattern frontier is converted into bounded investigations, how one investigation is decomposed into hypotheses and executable evidence, and how L1/L2 updates determine whether later budget is used to deepen, switch, or retire directions. The figure is intended as a process-level view of the framework.

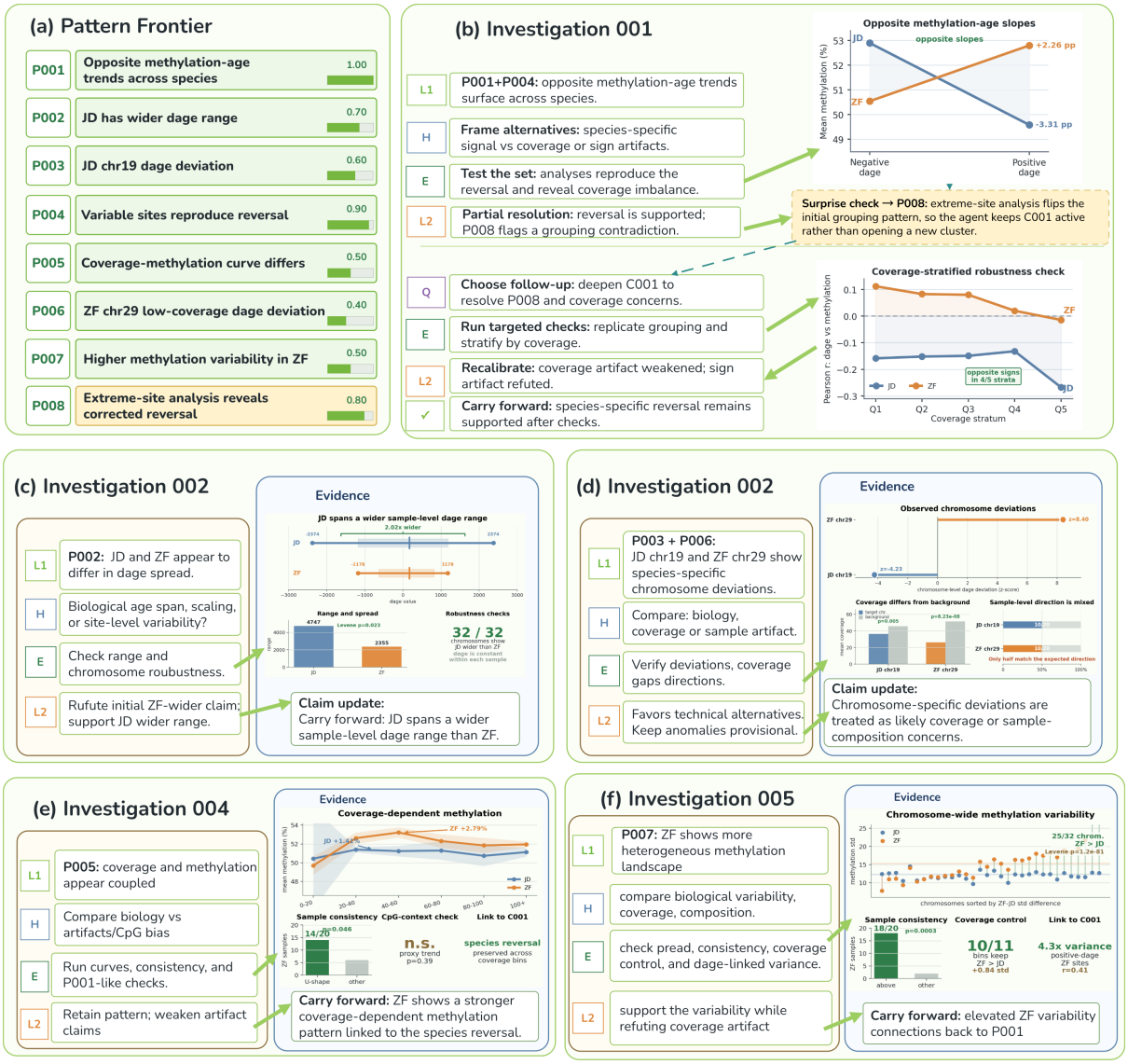


Figure 19. Discovery trace example for the BIX-52 case. The panel shows the prioritized pattern frontier, a detailed L1/L2 trace for Investigation 001, and summary views of subsequent investigations.